

All-Pairs Shortest Paths

Floyd-Warshall Algorithm — Design & Implementation in C

1. Aim

To design and implement a C program that solves the All-Pairs Shortest Paths (APSP) problem using Floyd's (Floyd-Warshall) algorithm, which computes the shortest path between every pair of vertices in a weighted directed graph.

2. Problem Description

Given a weighted directed graph $G = (V, E)$ with n vertices and a weight matrix W where $W[i][j]$ represents the weight of edge from vertex i to vertex j :

1. If there is no direct edge from i to j , $W[i][j] = \infty$ (infinity).
2. $W[i][i] = 0$ for all vertices i (distance from a vertex to itself is zero).
3. Goal: Find the shortest distance $d[i][j]$ between every pair (i, j) of vertices.

3. Algorithm (Floyd-Warshall)

3.1 Core Idea

Floyd's algorithm uses dynamic programming. It iteratively improves estimates of shortest paths by considering each vertex k as an intermediate node. The key recurrence is:

$$D(k)[i][j] = \min(D(k-1)[i][j], D(k-1)[i][k] + D(k-1)[k][j])$$

After k iterations, $D(k)[i][j]$ holds the length of the shortest path from i to j that uses only vertices $\{1, 2, \dots, k\}$ as intermediates.

3.2 Pseudocode

```
FLOYD(W, n)
    // Initialize distance matrix
    D ← W

    for k ← 1 to n do
        for i ← 1 to n do
            for j ← 1 to n do
                D[i][j] ← min(D[i][j], D[i][k] + D[k][j])
            end for
        end for
    end for

    return D    // D[i][j] = shortest path from i to j
```

3.3 Steps

4. Initialise the distance matrix D with the weight matrix W of the graph.
5. For each intermediate vertex k (from 1 to n):
6. For every source vertex i (from 1 to n):
7. For every destination vertex j (from 1 to n):
8. If $D[i][k] + D[k][j] < D[i][j]$, update $D[i][j]$.
9. After all n passes, $D[i][j]$ contains the shortest-path distance from i to j .
10. If $D[i][i] < 0$ for any vertex i , a negative-weight cycle is present.

4. Worked Example

4.1 Input Graph (4 Vertices)

Consider the following weighted directed graph (∞ = no direct edge):

D^0	v1	v2	v3	v4
v1	0	3	∞	7
v2	8	0	2	∞
v3	5	∞	0	1
v4	2	∞	∞	0

4.2 Matrix After Each Iteration

k = 1: Intermediate vertex v1 considered.

D1	v1	v2	v3	v4
v1	0	3	∞	7
v2	8	0	2	15
v3	5	8	0	1
v4	2	5	∞	0

k = 2: Intermediate vertex v2 considered.

D2	v1	v2	v3	v4
v1	0	3	5	7
v2	8	0	2	15
v3	5	8	0	1
v4	2	5	7	0

k = 3: Intermediate vertex v3 considered.

D3	v1	v2	v3	v4
v1	0	3	5	6
v2	7	0	2	3
v3	5	8	0	1
v4	2	5	7	0

k = 4: Intermediate vertex v4 considered.

D4	v1	v2	v3	v4
v1	0	3	5	6
v2	5	0	2	3
v3	3	6	0	1
v4	2	5	7	0

Complexity Analysis

Time Complexity: $O(V^3)$ — three nested loops each running V times.

Space Complexity: $O(V^2)$ — for the distance matrix (in-place update; no extra matrix needed).

Best Case: $O(V^3)$ — the triple loop always executes fully.

Works on: Graphs with negative-weight edges (but not negative-weight cycles).